CXL Memory Fault Injection Framework

This framework is a Python-based automation pipeline designed to test and validate CXL memory poisoning. It targets a CXL memory device, calculates aligned Device Physical Addresses (DPAs), injects media errors via the CCI, and verifies the hardware's internal error logging.

Prerequisites

- **Permissions:** root access (mandatory for interacting with kernel mailbox interfaces).
- **Kernel:** Version 6.0 or higher. Earlier kernels lack the native CXL mailbox drivers required to pass INJECT_POISON commands to the hardware layer.
- **Dependencies:** Python 3, ndctl toolkit (cxl-cli).

1. Overview

The pipeline is organized into six sequential phases.

Phase	Responsibility
1. Environment Check	Verify root access, Python availability, and kernel version support
2. Discovery	Query cxl-cli for attached CXL memory devices and build a topology map
3. Strategy	Compute cache-line-aligned addresses and select injection targets
4. Injection	Issue the poison-injection command at the chosen DPA via cxl-cli
5. Detection	Re-query the device's media-error list to confirm the fault was registered
6. Reporting	Aggregate every phase's output into a single structured JSON report

Phase 1 — Environment Check

The script checks for two things before it does anything else: root access (because you can't inject memory faults without admin privileges) and Kernel 6.0+ (because older kernels don't support the commands needed to send the poison payload).

Root and Dependency Checks

The script confirms it has the privileges and tools it needs. CXL poison injection ultimately goes through a kernel mailbox interface that enforces admin permissions, so root access is mandatory.

```
# root check
if [[ $EUID -ne 0 ]]; then
  fail "This script must be run as root."
  echo -e "  Try:  ${YELLOW}sudo $0 $*${RESET}"
  exit 1
fi
ok "Running as root"

# python check
if command -v python3 &>/dev/null; then
  ok "python3 found: $(python3 --version)"
else
  fail "python3 not found. Install it and retry."
  exit 1
fi
```

Kernel Version Checking

```
def check_kernel():
    print("\n-- checking kernel version")
    raw = platform.release(); info(f"kernel: {raw}")
    try:
        parts = raw.split("."); major, minor = int(parts[0]), int(parts[1])
    except (IndexError, ValueError):
        warn("could not parse kernel version, continuing anyway"); return "unknown", True
    ver = f"{major}.{minor}"
    if major > 6 or (major == 6 and minor >= 5):
        ok(f"kernel {ver} -- full cxl inject support"); return ver, True
    elif major == 6:
        warn(f"kernel {ver} -- partial support"); return ver, True
    else:
        fail(f"kernel {ver} -- too old, need 6.0+"); return ver, False
```

Kernel Range	CXL Support Level
< 5.12	No CXL core modules — devices are not visible to the OS
5.12 – 5.18	CXL 1.1 devices visible, but no mailbox driver or RAS support
6.0+	Mailbox commands work and support RAS

Phase 2 — Discovery (cxl list --memdevs)

The framework shells out to query attached devices (cxl list --memdevs) and builds an internal JSON topology map. By extracting the ram_size of the discovered hardware, the script defines the exact physical boundaries (dpa_end) to ensure all subsequent fault injections target valid memory regions on the device.

Querying Devices

```
def list_devices_via_cli():
    print("\n-- querying cxl-cli for memory devices")
    try:
        r = subprocess.run(["cxl", "list", "--memdevs"], capture_output=True, text=True,
        timeout=10)
        if r.returncode != 0 or not r.stdout.strip():
            warn("cxl-cli returned no devices")
            return []
        data = json.loads(r.stdout.strip())
        if isinstance(data, dict):
            data = [data]
        ok(f"cxl-cli found {len(data)} device(s)")
        return data
    except (FileNotFoundError, json.JSONDecodeError):
        warn("cxl-cli unavailable or returned invalid output")
        return []
```

Building the Topology Map

```
def build_topology(cli_devices):
    print("\n-- building topology map")
    devices = []
    total = 0
    for d in cli_devices:
        name = d.get("memdev")
        if not name:
            continue

        size = d.get("ram_size", 0)
        devices.append({
            "name": name,
            "dpa_start": "0x0",
            "dpa_end" : hex(size),
            "size_bytes": size,
            "size_gb": round(size/ (1024 ** 3), 2),
        })
        total+=size
    topology = {
        "total_devices": len(devices),
        "total_cxl_memory_gb": round(total/(1024**3),2),
        "devices": devices
    }
    ok(f"topology ready: {len(devices)} device(s), {topology['total_cxl_memory_gb']} GB total")
    return topology
```

Phase 3 — Strategy

This phase selects the specific DPAs to target.

- **The Alignment Rule:** CXL/PCIe specifications require memory operations to occur at cache-line granularity. The script utilizes an align() function to ensure every target address is strictly a multiple of 64 bytes. Unaligned addresses will be rejected by the hardware.
- **random_strategy:** Builds a pool of valid aligned addresses and selects one at random to simulate unpredictable memory degradation.
- **sweep:** Sequentially targets every valid cache-line aligned address for exhaustive validation.

Random Strategy

```
def random_strategy(device):
    info(f"strategy random on {device['name']}")
    start = int(device["dpa_start"], 16)
    end = int(device["dpa_end"], 16)
    pool = []
    addr = align(start + CACHELINE)
    while addr < end and len(pool) < 100_000:
        pool.append(addr)
        addr += CACHELINE
    chosen = random.choice(pool)
    ok(f"selected dpa: {hex(chosen)} on {device['name']}")
    return [{"device": device["name"], "dpa": hex(chosen), "strategy": "random_strategy"}]
```

This builds the pool of every cache-line-aligned address in the device's range (capped at 100,000 entries so a very large device doesn't exhaust memory just to build the list), then picks one at random. This is the default strategy and the one that gives every run a different injection target.

Sweep Strategy

```
def sweep(device, limit=None):
    info(f"strategy sweep on {device['name']}" + (f" (limit {limit})" if limit else ""))
    start = int(device["dpa_start"], 16)
    end = int(device["dpa_end"], 16)
    targets = []
    addr = align(start + CACHELINE)
    while addr < end:
        targets.append({"device": device["name"], "dpa": hex(addr), "strategy": "sweep"})
        addr += CACHELINE
        if limit and len(targets) >= limit:
            break
    ok(f"sweep generated {len(targets)} target(s) on {device['name']}")
    return targets
```

Sweep walks every aligned address in order and returns them all (or stops early if --limit is given). This is useful for exhaustive testing of a device, while random_strategy is closer to simulating an organic, unpredictable fault.

Phase 4 — Injection (cxl inject-media-poison <dev> -a <dpa>)

The selected DPA is passed to cxl inject-media-poison. This acts as a userspace wrapper that translates the CLI arguments into a raw CXL_MEM_COMMAND_INJECT_POISON payload. The kernel deposits this command directly into the CXL endpoint's mailbox registers.

Issuing the Injection

```
def inject_cli(device_name, dpa):
    info(f"injecting via cxl-cli: {device_name} @ {dpa}")
    try:
        r = subprocess.run(["cxl", "inject-media-poison", device_name, "-a", str(dpa)],
                           capture_output=True, text=True, timeout=10)
        if r.returncode == 0:
            ok("cxl-cli injection accepted")
            return True, "cxl-cli"
        warn(f"cxl-cli injection failed: {r.stderr.strip()}")
        return False, "cxl-cli"
    except FileNotFoundError:
        warn("cxl-cli not found")
        return False, "cxl-cli"
    except subprocess.TimeoutExpired:
        warn("cxl-cli injection timed out")
        return False, "cxl-cli"
```

Phase 5 — Detection (cxl list --media-errors -m <dev>)

Injection success at the kernel level does not guarantee the hardware updated its state. To verify, the script immediately re-queries the device's internal state (cxl list --media-errors). If the targeted DPA appears in the hardware-maintained poison list, the script registers a confirmed HIT.

Checking the Poison List

```
def detection_check(device_name, dpa):
    info(f"detection check on {device_name}")
    try:
        r = subprocess.run(["cxl", "list", "--media-errors", "-m", device_name],
                           capture_output=True, text=True, timeout=10)

        raw = r.stdout.strip()
        target_addr = to_int(dpa)

        data = json.loads(raw) if raw else []
        if isinstance(data, dict):
            data = [data]
        for dev in data:
            for err in dev.get("media_errors", []):
                if to_int(err.get("offset", -1)) == target_addr:
                    ok(f"poisoned memory found: {dpa}")
                    return { "result": "HIT",
                             "dpa_found": dpa }
        warn(f"poisoned memory not found: {dpa}")
        return { "result": "MISS",
                 "dpa_found": None }
    except Exception as e:
        warn(f"error -- {e}")
        return { "result": "ERROR",
                 "dpa_found": None }
```

Phase 6 — Reporting

The script aggregates the pass/fail results of every target into a structured JSON artifact. A target is strictly classified as an overall PASS only if the injection command was accepted (Phase 4) **and** the hardware logged the fault (Phase 5).

Building the Structured Report

```
def build_report(run_id, checks, topology, strategy, all_inj, all_det):
    results = []
    passed = 0
    failed = 0
    for inj, det in zip(all_inj, all_det):
        det_result = det["detection_check"]["result"]
        overall = "PASS" if inj["injected"] and det_result == "HIT" else "FAIL"
        results.append({
            "device": inj["device"], "dpa": inj["dpa"],
            "injection": {"method": inj.get("injection_method"),
                          "timestamp": inj.get("injection_time")},
            "detection": det["detection_check"],
            "overall": overall
        })
        if overall == "PASS":
            passed += 1
        else:
            failed += 1
    return {
        "run_id": run_id, "timestamp": time.strftime("%Y-%m-%dT%H:%M:%SZ", time.gmtime()),
        "environment": {"kernel": checks.get("kernel_version")},
        "topology": {"total_devices": topology["total_devices"]},
        "strategy": strategy, "results": results,
        "summary": {"total_targets": len(results), "passed": passed, "failed": failed}
    }
```

Output

```
  environment checks

-- checking kernel version
  [info] kernel: 6.18.0
  [pass] kernel 6.18 -- full cxl inject support

  checks done

  device discovery

-- querying cxl-cli for memory devices
  [pass] cxl-cli found 1 device(s)

-- building topology map
  [pass] topology ready: 1 device(s), 0.5 GB total

  discovery complete
{
  "total_devices": 1,
  "total_cxl_memory_gb": 0.5,
  "devices": [
    {
      "name": "mem0",
      "dpa_start": "0x0",
      "dpa_end": "0x20000000",
      "size_bytes": 536870912,
      "size_gb": 0.5
    }
  ]
}

  strategy engine
  [info] strategy random on mem0
  [pass] selected dpa: 0xb9dc0 on mem0
```

```
  injection and detection

  [1/1] mem0 @ 0xb9dc0

  target: mem0 @ 0xb9dc0
  [info] injecting via cxl-cli: mem0 @ 0xb9dc0
  [pass] cxl-cli injection accepted

  detection: mem0 @ 0xb9dc0
  [info] detection check on mem0
  [pass] poisoned memory found: 0xb9dc0

  result: mem0 @ 0xb9dc0
  ----------------------------------------
  [pass] injected via cxl-cli
  [pass] detection: 0xb9dc0 found in poison list
  ----------------------------------------
  overall: pass

  report

  run summary -- cxl-fi-20260629-045119
  ----------------------------------------
  strategy   : random_strategy
  kernel     : 6.18
  ----------------------------------------
  total   : 1
  passed  : 1
  failed  : 0
  ----------------------------------------
  all tests passed
```